

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Coding Challenge

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5 – Coding Challenge	Assessment:	Assessment Tool 1– Coding Challenge
Weightage:	40% of CIA	Total Marks:	100
CO5 Statement:	Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4)		
Student Name:	Abzam A	Reg. No.:	192573029
Section / Batch:	CSE - DS	Date:	27/08/26

Code of Conduct

I Abzam A (192573029) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Abzam A

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

DSA0404 – Fundamentals of Data Science
CO5-AT2 Coding Challenge

Q1. Design and implement a custom evaluation pipeline that compares kNN, Decision Tree, Logistic Regression, and K-Means-derived cluster labels (used as a weak classifier) on the same dataset using a statistically valid comparison —implement a paired t-test or McNemar’s test from scratch across cross-validation folds to determine whether the performance differences between models are statistically significant, not just numerically different.

Code:

```
import numpy as np

from math import sqrt, erf

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score


def normal_cdf(x):
    return 0.5 * (1 + erf(x / sqrt(2)))


def paired_t_test(scores1, scores2):
    differences = np.array(scores1) - np.array(scores2)
```

```
n = len(differences)
mean_difference = np.mean(differences)
std_difference = np.std(differences, ddof=1)

if std_difference == 0:
    t_statistic = 0
    p_value = 1.0
else:
    t_statistic = mean_difference / (std_difference / sqrt(n))
    p_value = 2 * (1 - normal_cdf(abs(t_statistic)))

return t_statistic, p_value
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
```

```
knn_scores = []
```

```
dt_scores = []
```

```
lr_scores = []
```

```
kmeans_scores = []
```

```
for fold, (train_index, test_index) in enumerate(cv.split(X, y), start=1):
```

```
X_train = X[train_index]
```

```
X_test = X[test_index]
```

```
y_train = y[train_index]
```

```
y_test = y[test_index]
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X_train_scaled, y_train)
```

```
knn_predictions = knn.predict(X_test_scaled)
```

```
knn_accuracy = accuracy_score(y_test, knn_predictions)
```

```
knn_scores.append(knn_accuracy)
```

```
decision_tree = DecisionTreeClassifier(random_state=42)
```

```
decision_tree.fit(X_train, y_train)
```

```
dt_predictions = decision_tree.predict(X_test)
```

```
dt_accuracy = accuracy_score(y_test, dt_predictions)
```

```
dt_scores.append(dt_accuracy)
```

```
logistic_regression = LogisticRegression(max_iter=5000, random_state=42)
```

```
logistic_regression.fit(X_train_scaled, y_train)
```

```
lr_predictions = logistic_regression.predict(X_test_scaled)
```

```
lr_accuracy = accuracy_score(y_test, lr_predictions)
```

```
lr_scores.append(lr_accuracy)
```

```

kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
kmeans.fit(X_train_scaled)

train_clusters = kmeans.predict(X_train_scaled)
cluster_to_class = {}

for cluster in range(2):
    cluster_labels = y_train[train_clusters == cluster]

    if len(cluster_labels) > 0:
        cluster_to_class[cluster] = np.bincount(cluster_labels).argmax()
    else:
        cluster_to_class[cluster] = 0

test_clusters = kmeans.predict(X_test_scaled)

kmeans_predictions = np.array(
    [cluster_to_class[cluster] for cluster in test_clusters]
)

kmeans_accuracy = accuracy_score(y_test, kmeans_predictions)
kmeans_scores.append(kmeans_accuracy)

print(f'Fold {fold}')
print("kNN Accuracy:", round(knn_accuracy, 4))
print("Decision Tree Accuracy:", round(dt_accuracy, 4))
print("Logistic Regression Accuracy:", round(lr_accuracy, 4))

```

```
print("K-Means Accuracy:", round(kmeans_accuracy, 4))  
print()
```

```
print("AVERAGE ACCURACY")  
print("kNN:", round(np.mean(knn_scores), 4))  
print("Decision Tree:", round(np.mean(dt_scores), 4))  
print("Logistic Regression:", round(np.mean(lr_scores), 4))  
print("K-Means:", round(np.mean(kmeans_scores), 4))
```

```
models = {  
    "kNN": knn_scores,  
    "Decision Tree": dt_scores,  
    "Logistic Regression": lr_scores,  
    "K-Means": kmeans_scores  
}
```

```
model_names = list(models.keys())
```

```
print("\nPAIRED T-TEST RESULTS")
```

```
for i in range(len(model_names)):  
    for j in range(i + 1, len(model_names)):  
  
        model1 = model_names[i]  
        model2 = model_names[j]
```

```

t_statistic, p_value = paired_t_test(
    models[model1],
    models[model2]
)

print("\n", model1, "vs", model2)
print("t-statistic:", round(t_statistic, 4))
print("p-value:", round(p_value, 4))

if p_value < 0.05:
    print("Result: Statistically Significant")
else:
    print("Result: Not Statistically Significant")

average_scores = {
    "kNN": np.mean(knn_scores),
    "Decision Tree": np.mean(dt_scores),
    "Logistic Regression": np.mean(lr_scores),
    "K-Means": np.mean(kmeans_scores)
}

best_model = max(average_scores, key=average_scores.get)

print("\nBEST MODEL:", best_model)
print("BEST ACCURACY:", round(average_scores[best_model], 4))

```


Output:

```
IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/irsha/Documents/DSA0405 programs/CO5-AT2-Coding Challenge.py
Fold 1
kNN Accuracy: 0.9825
Decision Tree Accuracy: 0.9123
Logistic Regression Accuracy: 0.9825
K-Means Accuracy: 0.9825

Fold 2
kNN Accuracy: 0.9825
Decision Tree Accuracy: 0.9298
Logistic Regression Accuracy: 0.9825
K-Means Accuracy: 0.9123

Fold 3
kNN Accuracy: 0.9649
Decision Tree Accuracy: 0.9649
Logistic Regression Accuracy: 0.9474
K-Means Accuracy: 0.9123

Fold 4
kNN Accuracy: 0.9474
Decision Tree Accuracy: 0.8947
Logistic Regression Accuracy: 0.9474
K-Means Accuracy: 0.8772

Fold 5
kNN Accuracy: 0.9474
Decision Tree Accuracy: 0.8947
Logistic Regression Accuracy: 0.9825
K-Means Accuracy: 0.9298

Fold 6
kNN Accuracy: 0.9474
Decision Tree Accuracy: 0.9649
Logistic Regression Accuracy: 0.9474
K-Means Accuracy: 0.9123

Fold 7
kNN Accuracy: 1.0
Decision Tree Accuracy: 0.9298
Logistic Regression Accuracy: 1.0
K-Means Accuracy: 0.8947

Fold 8
kNN Accuracy: 0.9825
Decision Tree Accuracy: 0.9123
Logistic Regression Accuracy: 0.9825
```

Geotag:

